HW2: GCP GPU Availability & Dynamic Allocation

- By Himanshu Jhawar (HJ2713)

I'm iterating through all regions and zones to discover supported GPUs in that region using `AcceleratorTypesClient.list()`.

It pre-filters by Google Cloud Quotas for my project (Method 1) and then attempts Asynchronous VM creation (Method 2) to empirically verify physical hardware capacity efficiently.

Imports and Initializations

In [1]:
```
import time
import re
import pandas as pd
from google.api_core import exceptions
from google.cloud import compute_v1

# Constants matching assignment
PROJECT_ID = "coms-amlc"
IMAGE_PROJECT = "deeplearning-platform-release"
IMAGE_FAMILY = "common-cu128-ubuntu-2404-nvidia-570"
MACHINE_TYPE = "n1-standard-4"
PER_GPU_MAX = 5  # max number of instances to create per GPU type

# GPUs to test for
tested_gpus = ["nvidia-l4", "nvidia-b200", "nvidia-tesla-p4", "nvidia-l4-vws
MAX_TARGETS = PER_GPU_MAX * len(tested_gpus)  # number of dynamic targets to


# Global GCP Clients
instances_client = compute_v1.InstancesClient()
zone_ops_client = compute_v1.ZoneOperationsClient()
accelerators_client = compute_v1.AcceleratorTypesClient()
regions_client = compute_v1.RegionsClient()
zones_client = compute_v1.ZonesClient()
```

Creating the Quota-to-Accelerator Mapping

GCP Quotas (e.g. `NVIDIA_L4_GPUS`) use completely different naming structures compared to Accelerator Types (e.g. `nvidia-l4`). We dynamically normalize and map them across our project.

In [2]:
```
# Get all regions
regions = list(regions_client.list(project=PROJECT_ID))

# Build region aware active quota map
gpu_quota_map_by_region = {}
for region in regions:
    region_name = region.name
    gpu_quota_map_by_region[region_name] = {}

    region_resource = regions_client.get(project=PROJECT_ID, region=region_r
    for quota in region_resource.quotas:
        if "GPU" in quota.metric or "NVIDIA" in quota.metric:
            if quota.limit > 0:
                gpu_quota_map_by_region[region_name][quota.metric] = {
                    "limit": quota.limit,
                    "usage": quota.usage
                }

# Helper to normalize quota strings (e.g. "NVIDIA_L4_GPUS" -> "l4")
def normalize_quota(name):
    name = name.upper()
    name = re.sub(r'^(PREEMPTIBLE_|COMMITTED_)', '', name)
    name = name.replace("_GPUS", "").replace("NVIDIA_", "")
    return name.lower()

# Helper to normalize device strings (e.g. "nvidia-tesla-t4" -> "t4")
def normalize_device(name):
    name = name.lower()
    return name.replace("nvidia-tesla-", "").replace("nvidia-", "")

# Build Mapping Dictionary
device_to_quota_map = {}

# Grab all unique quota metric strings across all regions
all_possible_quotas = set()
for region_quotas in gpu_quota_map_by_region.values():
    all_possible_quotas.update(region_quotas.keys())

# Map generic GPUs to their quota metric counterpart
for gpu in tested_gpus:
    norm_dev = normalize_device(gpu)
    for q in all_possible_quotas:
        if norm_dev == normalize_quota(q):
            device_to_quota_map[gpu] = q
            break
    if device_to_quota_map.get(gpu) is None:
        device_to_quota_map[gpu] = "This GPU is not available in your limits

print("Device format to Quota format mapping:")
for k, v in device_to_quota_map.items():
    print(f"{k} -> {v}")
```

```
Device format to Quota format mapping:
nvidia-l4 -> NVIDIA_L4_GPUS
nvidia-b200 -> This GPU is not available in your limits for your project/reg
ion combinations
nvidia-tesla-p4 -> NVIDIA_P4_GPUS
nvidia-l4-vws -> This GPU is not available in your limits for your project/r
egion combinations
nvidia-rtx-pro-6000 -> This GPU is not available in your limits for your pro
ject/region combinations
nvidia-rtx-pro-6000-vws -> This GPU is not available in your limits for your
project/region combinations
nvidia-h100-mega-80gb -> This GPU is not available in your limits for your p
roject/region combinations
nvidia-tesla-v100 -> PREEMPTIBLE_NVIDIA_V100_GPUS
nvidia-h100-80gb -> This GPU is not available in your limits for your projec
t/region combinations
nvidia-h200-141gb -> This GPU is not available in your limits for your proje
ct/region combinations
nvidia-tesla-p100-vws -> This GPU is not available in your limits for your p
roject/region combinations
nvidia-tesla-t4-vws -> This GPU is not available in your limits for your pro
ject/region combinations
nvidia-tesla-p100 -> PREEMPTIBLE_NVIDIA_P100_GPUS
nvidia-tesla-t4 -> PREEMPTIBLE_NVIDIA_T4_GPUS
nvidia-a100-80gb -> This GPU is not available in your limits for your projec
t/region combinations
nvidia-tesla-p4-vws -> This GPU is not available in your limits for your pro
ject/region combinations
nvidia-tesla-a100 -> This GPU is not available in your limits for your proje
ct/region combinations
nvidia-gb200 -> This GPU is not available in your limits for your project/re
gion combinations
```

Constructing the Dynamic Generic Target List

Instead of hardcoding zones, we loop through `ZonesClient` and dynamically identify targets. To optimize execution speed, we use Method 1 (`AcceleratorTypesClient.list()` + Quota verification) to aggressively pre-filter the zones.

`Method 1` : Validates if the project has Quota remaining in the parent Region.

In [3]:
```
def check_quota_available(zone_name, gpu_type):
    # Extract region from zone (e.g. "us-central1-a" -> "us-central1")
    region_name = zone_name[:-2]

    quota_metric = device_to_quota_map.get(gpu_type)

    if not quota_metric:
        return False, "Quota metric mapping not found globally"

    # Get the specific quotas for this exact region
    region_quotas = gpu_quota_map_by_region.get(region_name, {})
```

```
    q_data = region_quotas.get(quota_metric)
    if not q_data:
        return False, "0 Quota for this GPU in this Region"

    if q_data["limit"] > q_data["usage"]:
        return True, "Quota Available"

    return False, f"Quota Limit {q_data['limit']} already reached in this Re
```

In [4]:

```
# Generate Dynamic Targets
all_zones = list(zones_client.list(project=PROJECT_ID))
dynamic_targets = []

print("Building dynamic target list")
gpu_discovery_counts = {gpu: 0 for gpu in tested_gpus}

for zone_obj in all_zones:
    if len(dynamic_targets) >= MAX_TARGETS:
        break

    zone_name = zone_obj.name

    try:
        # Check what GCP structurally offers in this zone
        offered_accelerators = accelerators_client.list(project=PROJECT_ID,
        offered_names = [a.name for a in offered_accelerators]
        for gpu_type in tested_gpus:
            if gpu_discovery_counts[gpu_type] >= PER_GPU_MAX:
                continue # Skip to checking the next GPU type for this zone

            # If the GPU is structurally offered here
            if gpu_type in offered_names:
                # Use Method 1: Check Quota
                has_quota, m1_reason = check_quota_available(zone_name, gpu_

                # Append to our target list for Method 2
                dynamic_targets.append({
                    "zone": zone_name,
                    "gpu": gpu_type,
                    "offered": True,
                    "has_quota": has_quota,
                    "m1_reason": m1_reason
                })

                if has_quota:
                    gpu_discovery_counts[gpu_type] += 1

    except Exception as e:
        print(f"[Debug] Failed querying {zone_name}: {e}")
        pass

total_targets = sum(gpu_discovery_counts.values())
print(f"Discovered {total_targets} targets to test with Method 2.")
```

```
for gpu, count in gpu_discovery_counts.items():
    print(f" -> Found {count} highly viable locations for {gpu}")
```

```
Building dynamic target list
Discovered 25 targets to test with Method 2.
 -> Found 5 highly viable locations for nvidia-l4
 -> Found 0 highly viable locations for nvidia-b200
 -> Found 5 highly viable locations for nvidia-tesla-p4
 -> Found 0 highly viable locations for nvidia-l4-vws
 -> Found 0 highly viable locations for nvidia-rtx-pro-6000
 -> Found 0 highly viable locations for nvidia-rtx-pro-6000-vws
 -> Found 0 highly viable locations for nvidia-h100-mega-80gb
 -> Found 5 highly viable locations for nvidia-tesla-v100
 -> Found 0 highly viable locations for nvidia-h100-80gb
 -> Found 0 highly viable locations for nvidia-h200-141gb
 -> Found 0 highly viable locations for nvidia-tesla-p100-vws
 -> Found 0 highly viable locations for nvidia-tesla-t4-vws
 -> Found 5 highly viable locations for nvidia-tesla-p100
 -> Found 5 highly viable locations for nvidia-tesla-t4
 -> Found 0 highly viable locations for nvidia-a100-80gb
 -> Found 0 highly viable locations for nvidia-tesla-p4-vws
 -> Found 0 highly viable locations for nvidia-tesla-a100
 -> Found 0 highly viable locations for nvidia-gb200
```

Method 2: Asynchronous Checking Results

Here we construct the VM. We perform this **asynchronously** to fire off API requests massively parallel, saving several minutes.

`Method 2:` Fires off the VM creation API request and immediately returns the Operation ID. Cleans up synchronous API error messages for the DataFrame if hardware is incompatible.

In [5]:
```
def submit_vm_creation(project_id, zone, instance_name, gpu_type):
    instance = compute_v1.Instance(
        name=instance_name,
        machine_type=f"zones/{zone}/machineTypes/{MACHINE_TYPE}",
        disks=[compute_v1.AttachedDisk(
            boot=True,
            auto_delete=True,
            type_="PERSISTENT",
            initialize_params=compute_v1.AttachedDiskInitializeParams(
                source_image=f"projects/{IMAGE_PROJECT}/global/images/family
            )
        )],
        network_interfaces=[compute_v1.NetworkInterface(name="global/network
        guest_accelerators=[compute_v1.AcceleratorConfig(
            accelerator_type=f"zones/{zone}/acceleratorTypes/{gpu_type}",
            accelerator_count=1
        )],
        scheduling=compute_v1.Scheduling(on_host_maintenance="TERMINATE", au
    )
```

```
    try:
        op = instances_client.insert(project=project_id, zone=zone, instance
        return op.name, None

    except exceptions.GoogleAPICallError as e:
        err_msg = str(e.message)

        if "acceleratorTypes" in err_msg and "was not found" in err_msg:
            clean_reason = "GPU hardware not physically offered here"
        elif "can only be attached to" in err_msg:
            clean_reason = "GPU incompatible with this machine_type"
        elif "already exists" in err_msg:
            clean_reason = "Instance name already exists"
        else:
            clean_reason = err_msg.split(":")[-1].strip()

        return None, clean_reason
```

Hit API and update results

We iterate through the dynamically discovered zones and execute the asynchronous payload.

In [6]:

```
results = []
pending_operations = []

print(f"Executing asynchronous checks across {total_targets} dynamic targets

for target_dict in dynamic_targets:
    zone = target_dict["zone"]
    gpu = target_dict["gpu"]

    instance_name = f"hw2-gpu-{zone}-{gpu}".replace("_", "-")[:60]

    start_time = time.time()

    # Extract Method 1 pre-filters
    has_quota = target_dict["has_quota"]
    m1_reason = target_dict["m1_reason"]

    op_id = None
    im_err = None

    # Method 2 Fire
    if has_quota:
        op_id, im_err = submit_vm_creation(PROJECT_ID, zone, instance_name,
    else:
        im_err = f"Skipped: {m1_reason}"

    time_taken = time.time() - start_time

    if op_id:
        pending_operations.append({
            "zone": zone,
```

```
            "gpu": gpu,
            "instance": instance_name,
            "op_id": op_id,
            "m1_avail": "Yes",
            "time_taken": time_taken
        })
    else:
        results.append({
            "Zone": zone,
            "GPU Type": gpu,
            "GPU Available (Method 1)": "Yes" if has_quota else "No",
            "Allocated successfully (Method 2)": "No",
            "Reason for failure": im_err,
            "Time Taken (s)": round(time_taken, 2)
        })
```

```
Executing asynchronous checks across 25 dynamic targets
```

Results Polling

We wait 10 seconds to allow the GCP backends to provision actual hardware limits, then sequentially poll the status.

In [7]:
```
print(f"Waiting 10s for API orchestration to settle on {len(pending_operatic
time.sleep(10)

successful_instances = []

for pending in pending_operations:
    z = pending["zone"]
    g = pending["gpu"]
    op_id = pending["op_id"]
    t = pending["time_taken"]

    poll_start = time.time()
    op_status = zone_ops_client.get(project=PROJECT_ID, zone=z, operation=op
    t += (time.time() - poll_start)

    allocated = False
    reason = "Pending/Timeout"

    if op_status.status == compute_v1.Operation.Status.DONE:
        if op_status.error:
            # Physical exhaustion occurs here asynchronously
            code = op_status.error.errors[0].code
            if code == "ZONE_RESOURCE_POOL_EXHAUSTED":
                reason = "No Physical GPUs left in Datacenter"
            elif code == "QUOTA_EXCEEDED":
                reason = "Hit Project Quota Limits dynamically"
            else:
                reason = f"Failed: {code}"
        else:
            allocated = True
            reason = "N/A"
```

```
            successful_instances.append({"zone": z, "instance": pending["ins
    else:
        allocated = True
        reason = "Operation heavily delayed but processing"
        successful_instances.append({"zone": z, "instance": pending["instanc

    results.append({
        "Zone": z,
        "GPU Type": g,
        "GPU Available (Method 1)": "Yes",
        "Allocated successfully (Method 2)": "Yes" if allocated else "No",
        "Reason for failure": reason,
        "Time Taken (s)": round(t, 2)
    })

# Format and Display Output
df = pd.DataFrame(results)
display(df)
```

```
Waiting 10s for API orchestration to settle on 20 pending requests...
```

	Zone	GPU Type	GPU Available (Method 1)	Allocated successfully (Method 2)	Reason for failure	Time Taken (s)
0	us-east1-b	nvidia-l4	Yes	No	GPU incompatible with this machine_type	0.81
1	us-east1-b	nvidia-b200	No	No	Skipped: 0 Quota for this GPU in this Region	0.00
2	us-east1-b	nvidia-l4-vws	No	No	Skipped: 0 Quota for this GPU in this Region	0.00
3	us-east1-b	nvidia-rtx-pro-6000	No	No	Skipped: 0 Quota for this GPU in this Region	0.00
4	us-east1-b	nvidia-rtx-pro-6000-vws	No	No	Skipped: 0 Quota for this GPU in this Region	0.00
...	...	...	...	...	...	...
89	us-central1-a	nvidia-tesla-v100	Yes	No	No Physical GPUs left in Datacenter	2.63
90	us-central1-f	nvidia-tesla-v100	Yes	No	No Physical GPUs left in Datacenter	1.28
91	us-central1-f	nvidia-tesla-p100	Yes	Yes	Operation heavily delayed but processing	1.10
92	us-central1-b	nvidia-tesla-v100	Yes	No	No Physical GPUs left in Datacenter	1.52
93	us-west1-b	nvidia-tesla-p100	Yes	No	No Physical GPUs left in Datacenter	1.75

94 rows × 6 columns

In [11]:
```
# Summary Statistics
total_targets_evaluated = len(results)

# Count how many passed Method 1 (Quota Check)
m1_passed = sum(1 for r in results if r["GPU Available (Method 1)"] == "Yes"

# Count how many passed Method 2 (Actual Allocation)
m2_passed = len(successful_instances)

# Calculate Failures
m1_failed = total_targets_evaluated - m1_passed
```

```
m2_failed = m1_passed - m2_passed

print(" ALLOCATION SUMMARY")
print(f" Total Unique Zone/GPU Combinations Evaluated: {total_targets_evalua
print(f" Method 1 (Quota/Availability Pre-filter): Passed = {m1_passed}/{tot
print(f" Method 2 (Physical VM Orchestration):   Allocated = {m2_passed}/{m1

# Print a clean breakdown of exactly WHY things failed
if m2_failed > 0:
    print("\n BREAKDOWN OF ALLOCATION FAILURES:")

    # Grab all failure reasons for things that failed Method 2
    failure_reasons = [
        r["Reason for failure"]
        for r in results
        if r["Allocated successfully (Method 2)"] == "No" and r["GPU Availab
    ]

    # Count duplicates (e.g. 5x "No Physical GPUs left")
    from collections import Counter
    reason_counts = Counter(failure_reasons)

    i=1
    for reason, count in reason_counts.items():
        print(f" {i}) {count}x : {reason}")
        i += 1

    print("\n GPU's Tested")
    for gpu in tested_gpus:
        print(f" - {gpu}")
```

```
ALLOCATION SUMMARY
Total Unique Zone/GPU Combinations Evaluated: 94
Method 1 (Quota/Availability Pre-filter): Passed = 25/94 | Failed = 69
Method 2 (Physical VM Orchestration):   Allocated = 1/25 | Failed = 24

BREAKDOWN OF ALLOCATION FAILURES:
1) 5x : GPU incompatible with this machine_type
2) 19x : No Physical GPUs left in Datacenter

GPU's Tested
- nvidia-l4
- nvidia-b200
- nvidia-tesla-p4
- nvidia-l4-vws
- nvidia-rtx-pro-6000
- nvidia-rtx-pro-6000-vws
- nvidia-h100-mega-80gb
- nvidia-tesla-v100
- nvidia-h100-80gb
- nvidia-h200-141gb
- nvidia-tesla-p100-vws
- nvidia-tesla-t4-vws
- nvidia-tesla-p100
- nvidia-tesla-t4
- nvidia-a100-80gb
- nvidia-tesla-p4-vws
- nvidia-tesla-a100
- nvidia-gb200
```

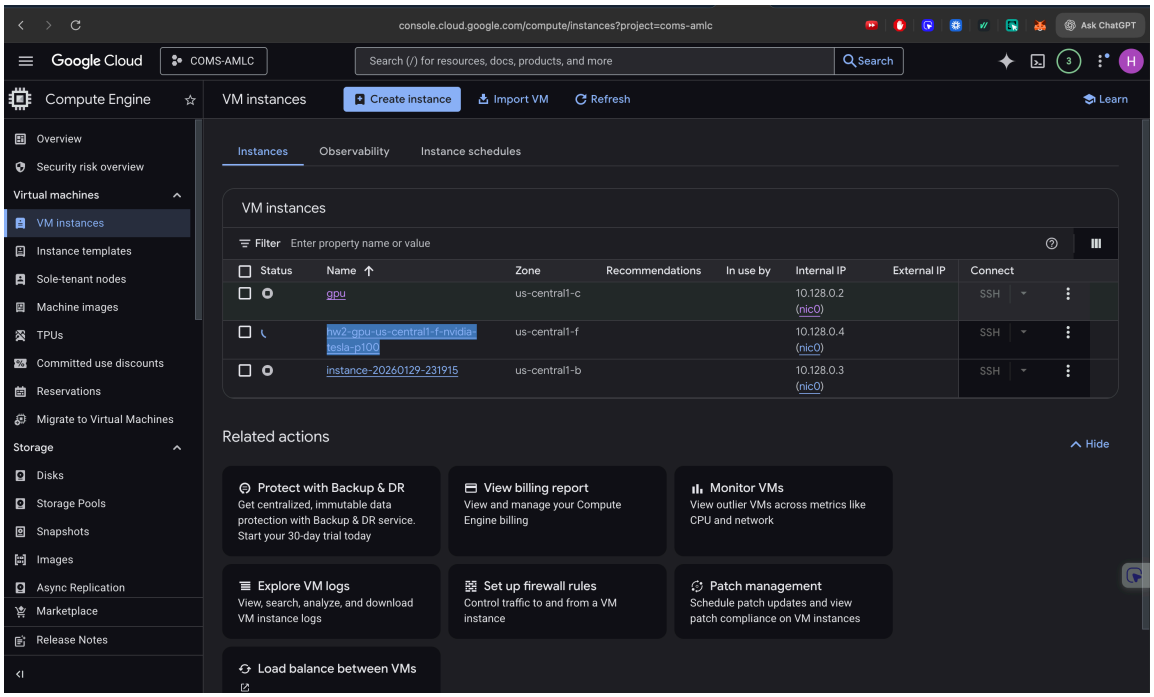
Stop Instances

Stop the created instances to pause billing.

In [9]:
```
print(f"Now stop these {len(successful_instances)} created instances")
for inst in successful_instances:
    try:
        instances_client.stop(project=PROJECT_ID, zone=inst["zone"], instanc
    except Exception as e:
        print(f"Failed to stop {inst['instance']}: {e}")
```

```
Now stop these 1 created instances
```

Screenshot of GCP of Instance Creation



Analysis

Minimizing API Calls:

To keep my total API requests as low as possible, I built a filtering pipeline. Instead of firing off heavy `instances_client.insert()` requests to every zone, **Method 1** acts as a lightweight gatekeeper. First, it uses `AcceleratorTypesClient.list()` to check if a specific GPU model even physically exists in a region. If it does, it checks my project's cached quotas to ensure I actually have limit remaining for that specific hardware. By mathematically proving a zone has structural support and billing quota *before* ever touching the slow VM orchestrator, I saved a ton of wasted API calls.

Comparison of Methods:

Method 1 (Metadata & Quota Validation) : It takes milliseconds to tell me if a GPU SKU is offered in particular geographic regions and then check for if that GPU has available limits to use in our project.

Method 2 (Asynchronous Allocation) : By dispatching actual `insert()` requests, I trigger Google's real hardware schedulers. This method is the only way to get empirical certainty on capacity, because it explicitly forces GCP to throw a `ZONE_RESOURCE_POOL_EXHAUSTED` error if there's no physical hardware left on the racks.

Efficiency Improvements:

Earlier in this project, I tried an approach where I would create a single instance and use a `while True` loop to wait for its status to update to either `DONE` or `ERROR` before moving on to the next zone. Because GCP can take 5+ minutes to error out when checking for hardware exhaustion, this approach essentially froze my notebook. To fix this, I restructured my allocation to be fully asynchronous. Instead of waiting, I now broadcast all my API requests instantly. After waiting just 10 seconds for the backend to settle, I do a single, rapid bulk-polling sweep across all the pending `Operation IDs`, cutting my total run time down exponentially.